Software Engineering Lab-6

SRN: PES1UG24CS102

Name: Ayaan Ahmed

Section: B

Branch coverage

```
C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6>.\.venv\Scripts\python.exe run_coverage.py branch
LAB 6 - BRANCH COVERAGE
Python 3.12.10

LAB6> python -m coverage run --data-file=.coverage.branch --branch --source=laundry_service -m pytest test_branch_coverage.py -q
.....................................................................                                    [100%]
============================================= warnings summary =============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: ca
che could not write path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.pytest_cache\v\cache\nodeids: [Errno 13] Permission
denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB6\\.pytest_cache\\v\\cache\\nodeids'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
71 passed, 1 warning in 0.18s
LAB6> python -m coverage report --data-file=.coverage.branch --show-missing --fail-under=90
Name                Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------
laundry_service.py      37      0     26      0   100%
---------------------------------------------------------------
TOTAL                   37      0     26      0   100%
LAB6> python -m coverage json --data-file=.coverage.branch --pretty-print -o reports/branch_coverage.json
Wrote JSON report to reports/branch_coverage.json

Statement coverage: 100.00% (37/37 statements)
Branch coverage: 100.00% (26/26 branch outcomes)
PASS - minimum required: 90%
```

Code coverage

```
C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6>.\.venv\Scripts\python.exe run_coverage.py code
LAB 6 - CODE (STATEMENT) COVERAGE
Python 3.12.10

LAB6> python -m coverage run --data-file=.coverage.code --source=laundry_service -m pytest test_code_coverage.py -q
....................                                                                                     [100%]
============================================= warnings summary =============================================
.venv\Lib\site-packages\_pytest\cacheprovider.py:487
  C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.venv\Lib\site-packages\_pytest\cacheprovider.py:487: PytestCacheWarning: ca
che could not write path C:\Users\Zia Ahmed\Downloads\3rd_Year\SE-labs\LAB6\.pytest_cache\v\cache\nodeids: [Errno 13] Permission
denied: 'C:\\Users\\Zia Ahmed\\Downloads\\3rd_Year\\SE-labs\\LAB6\\.pytest_cache\\v\\cache\\nodeids'
    config.cache.set("cache/nodeids", sorted(str(n) for n in self.cached_nodeids))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
20 passed, 1 warning in 0.06s
LAB6> python -m coverage report --data-file=.coverage.code --show-missing --fail-under=90
Name                Stmts   Miss  Cover   Missing
-----------------------------------------------
laundry_service.py      37      0   100%
-----------------------------------------------
TOTAL                   37      0   100%
LAB6> python -m coverage json --data-file=.coverage.code --pretty-print -o reports/code_coverage.json
Wrote JSON report to reports/code_coverage.json

Statement coverage: 100.00% (37/37 statements)
PASS - minimum required: 90%
```

Test_branch_coverage.py

```
"""Branch coverage: independently exercise decisions and their boundaries."""


import pytest


from laundry_service import (
    dropoff_status,
    extra_weight_charge,
    order_acceptable,
)



@pytest.mark.parametrize("garment", ["silk", "", None])
def test_unknown_garment_branch(garment):
    with pytest.raises(ValueError, match="^unknown garment type$"):
        extra_weight_charge(garment, 0, "standard", False)



@pytest.mark.parametrize("weight", [-0.01, -1, -100])
def test_negative_weight_branch(weight):
    with pytest.raises(ValueError, match="^invalid weight$"):
        extra_weight_charge("regular", weight, "standard", False)



def test_garment_validation_precedes_weight_validation():
    with pytest.raises(ValueError, match="^unknown garment type$"):
```

```
        extra_weight_charge("silk", -1, "standard", False)



@pytest.mark.parametrize(
    "garment, member, expected",
    [
        ("regular", False, 90),
        ("regular", True, 30),
        ("delicate", False, 150),
        ("delicate", True, 90),
        ("bedding", False, 0),
        ("bedding", True, 0),
    ],
)
def test_membership_condition_combinations(garment, member, expected):
    # Covers both short-circuit outcomes and the bedding membership exception.
    assert extra_weight_charge(garment, 8, "standard", member) == expected



@pytest.mark.parametrize(
    "garment, member, allowance",
    [
        ("regular", False, 5),
        ("regular", True, 7),
        ("delicate", False, 3),
        ("delicate", True, 5),
```

```
        ("bedding", False, 8),
        ("bedding", True, 8),
    ],
)
@pytest.mark.parametrize("extra, expected", [(-0.5, 0), (0, 0), (0.5, 15)])
def test_included_weight_boundary(garment, member, allowance, extra, expected):
    assert extra_weight_charge(
        garment, allowance + extra, "standard", member
    ) == expected



@pytest.mark.parametrize(
    "weight, speed, expected",
    [
        (0, "standard", 0),
        (5.5, "standard", 15),
        (5.5, "express", 25),
        (15, "standard", 300),
        (15, "express", 500),
        (15.5, "standard", 472.5),
        (15.5, "express", 787.5),
    ],
)
def test_rate_and_ten_extra_kilo_boundary(weight, speed, expected):
    # Exactly ten extra kg has no surcharge; above ten applies it to all extra kg.
    assert extra_weight_charge("regular", weight, speed, False) == pytest.approx(
```

```
        expected
    )


@pytest.mark.parametrize(
    "weight, items, expected",
    [(1, 4, True), (1, 5, True), (1, 6, False), (100, 5, True)],
)
def test_delicate_item_limit_and_weight_bypass(weight, items, expected):
    # Current specification checks only item count for delicate orders.
    assert order_acceptable(weight, items, True) is expected


@pytest.mark.parametrize(
    "weight, items, expected",
    [
        (24.99, 39, True),
        (24.99, 40, True),
        (24.99, 41, False),
        (25, 39, True),
        (25, 40, True),
        (25, 41, False),
        (25.01, 39, False),
        (25.01, 40, False),
        (25.01, 41, False),
    ],
```

```
)
def test_regular_weight_and_item_boundaries(weight, items, expected):
    assert order_acceptable(weight, items, False) is expected



@pytest.mark.parametrize(
    "bedding, express, cutoff",
    [(False, False, 60), (False, True, 90), (True, False, 90), (True, True, 120)],
)
@pytest.mark.parametrize(
    "offset, expected",
    [(1, "accepted"), (0, "accepted"), (-1, "late"), (-20, "late"), (-21, "rejected")],
)
def test_all_dropoff_decisions_and_cutoff_boundaries(
    bedding, express, cutoff, offset, expected
):
    assert dropoff_status(cutoff + offset, bedding, express) == expected
```

Test_code_coverage.py

```
"""Statement coverage: exercise every executable line of laundry_service."""

import pytest

from laundry_service import (
    dropoff_status,
    extra_weight_charge,
    order_acceptable,
)


def test_unknown_garment_is_rejected():
    with pytest.raises(ValueError, match="^unknown garment type$"):
        extra_weight_charge("leather", 2, "standard", False)


def test_negative_weight_is_rejected():
    with pytest.raises(ValueError, match="^invalid weight$"):
        extra_weight_charge("regular", -1, "standard", False)


def test_weight_inside_allowance_has_no_charge():
```

```
    assert extra_weight_charge("regular", 4, "standard", False) == 0


def test_membership_adds_two_kilos_for_delicates():
    assert extra_weight_charge("delicate", 5, "express", True) == 0


def test_standard_extra_weight_charge():
    assert extra_weight_charge("regular", 7, "standard", False) == 60


def test_express_extra_weight_charge():
    assert extra_weight_charge("regular", 7, "express", False) == 100


def test_fractional_extra_weight_charge():
    assert extra_weight_charge("regular", 5.5, "standard", False) == 15


def test_large_standard_order_uses_surcharge():
    # 16 - 5 = 11 extra kg; standard rate becomes 30 + 15.
    assert extra_weight_charge("regular", 16, "standard", False) == 495
```

```
def test_large_express_order_uses_both_surcharges():
    # 16 - 5 = 11 extra kg; express rate becomes 50 + 15 + 10.
    assert extra_weight_charge("regular", 16, "express", False) == 825


def test_bedding_does_not_receive_membership_allowance():
    assert extra_weight_charge("bedding", 9, "standard", True) == 30


@pytest.mark.parametrize(
    "weight, items, delicate, expected",
    [
        (3, 5, True, True),
        (3, 6, True, False),
        (26, 1, False, False),
        (25, 40, False, True),
        (10, 41, False, False),
    ],
)
def test_order_acceptance(weight, items, delicate, expected):
    assert order_acceptable(weight, items, delicate) is expected
```

```
@pytest.mark.parametrize(
    "minutes, bedding, express, expected",
    [
        (90, True, False, "accepted"),
        (90, False, True, "accepted"),
        (60, False, False, "accepted"),
        (45, False, False, "late"),
        (39, False, False, "rejected"),
    ],
)
def test_dropoff_outcomes(minutes, bedding, express, expected):
    assert dropoff_status(minutes, bedding, express) == expected
```